

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

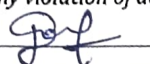
ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	Bhuvaneshwaram J	Reg. No.:	192511402
Section / Batch:	Slot - B	Date:	02/09/2026

Code of Conduct

I Bhuvaneshwaram J (192511402) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

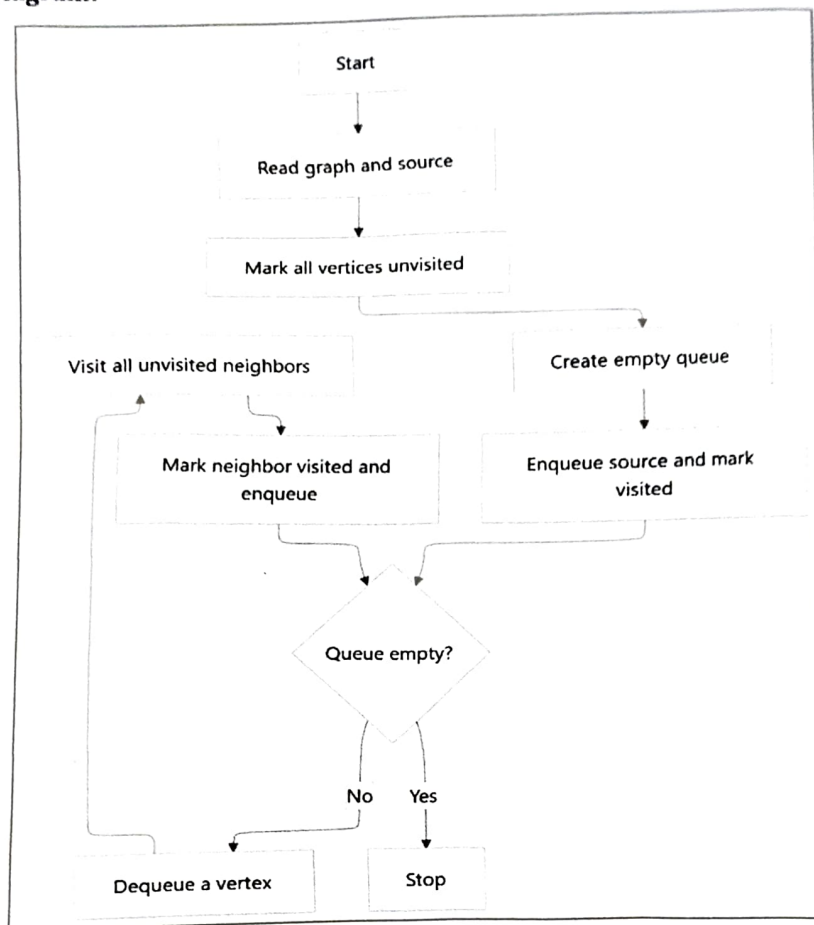
Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A - Flowchart-To-Code Conversion

1. Convert the flowchart for Breadth First Search traversal into code.

(20 Marks)

Flowchart Diagram:



Flowchart to code Visualization

Breadth First Search (BFS):

Code:

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int n, a[10][10], v[10] = {0}, q[0], r=0, i, x
```

```
    printf("vertices");
```

```
    scanf("%d", &n);
```

```
    printf("Adjacency matrix = \n");
```

```
    for (i=0; i<n; i++)
```

```
        for (x=0; x<n; x++) scanf("%d", &a[i][x]);
```

```
    printf("Source");
```

```
    scanf("%d", &s);
```

```
    a[r+s] = s, v[s] = 1;
```

```
    while (p < r) {
```

```
        x = q[p++];
```

```
        printf("%d", x);
```

```
        for (i=0; i<n; i++)
```

```
            if (a[x][i] && !v[i])
```

```
                v[i] = 1, q[r++] = i;
```

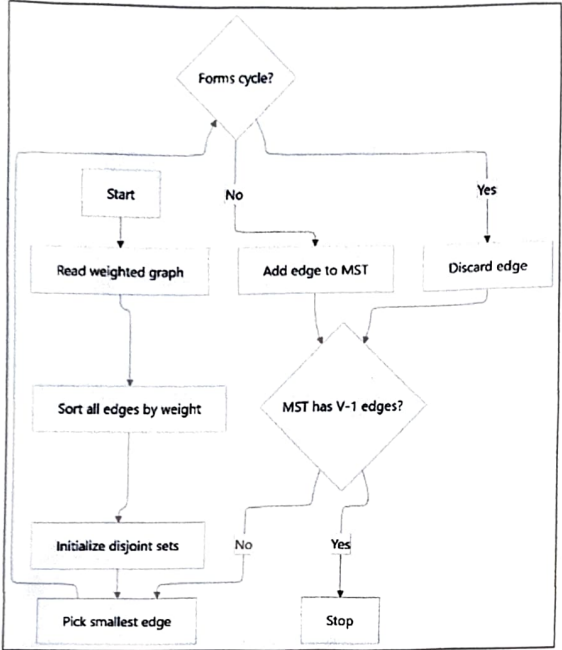
```
    }
```

```
    return 0;
```

```
}
```

2. Convert the flowchart for Kruskal's algorithm into code. (20 Marks)

Flowchart Diagram:



Kruskal's Algor

Code:

```
#include <stdio.h>
typedef struct {
    int u, v, w;
} Edge;
int p[20];
int find (int x) {
    return p[x] == x ? x : (p[x] = find(p[x]));
}
int main () {
    Edge e[30], t;
    int n, m, i, j, c = 0;
    scanf ("%d %d", &n, &m);
    for (i = 0; i < m; i++)
        scanf ("%d %d %d", &e[i].u, &e[i].v);
    for (i = 0; i < n; i++) p[i] = i;
    for (i = 0; i < m-1; i++)
        for (j = i+1; j < m; j++)
            if (e[i].w > e[j].w)
                t = e[i], e[i] = e[j], e[j] = t;
```

if (a == b)

{

p[a] = b;

printf("%d %d ; %d\n", e[i], v[e[i]], v[e[i]]);

c++;

}

}

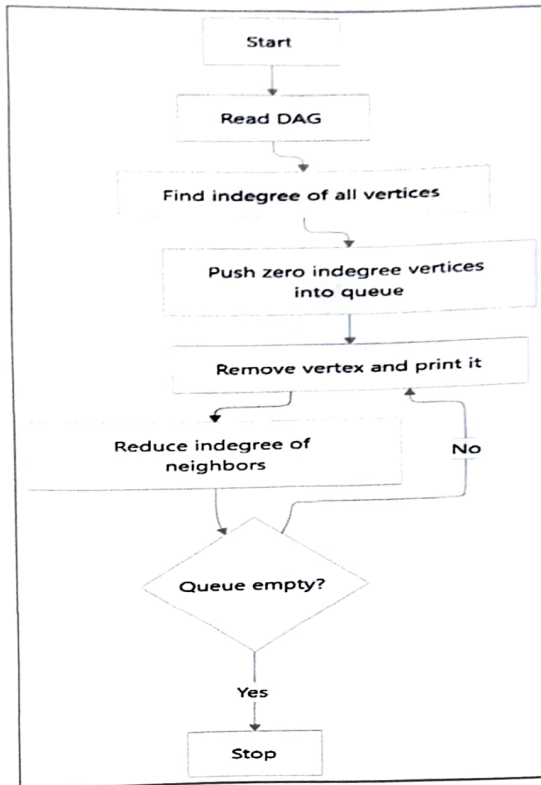
return 0;

}

3. Convert the flowchart for topological sort into code.

(20 Marks)

Flowchart Diagram:



Topological Sort:

Code:

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int n, a[10][10]; int[10] = {0}, r=0, x=0
```

```
    scanf ("%d", &n);
```

```
    for (i=0; i<n; i++)
```

```
        for (j=0; j<n; j++) {
```

```
            scanf ("%d", &a[i][j]);
```

```
            if (a[i][j]) in[j]++;
```

```
        }
```

```
    for (i=0; i<n; i++)
```

```
        if (in[i] == 0) r[r++] = i;
```

```
    while (r < n) {
```

```
        printf ("%d", x);
```

```
        for (i=0; i<n; i++)
```

```
            if (a[x][i] > 0 && in[i] == 0)
```

```
                r[r++] = i;
```

```
    }
```

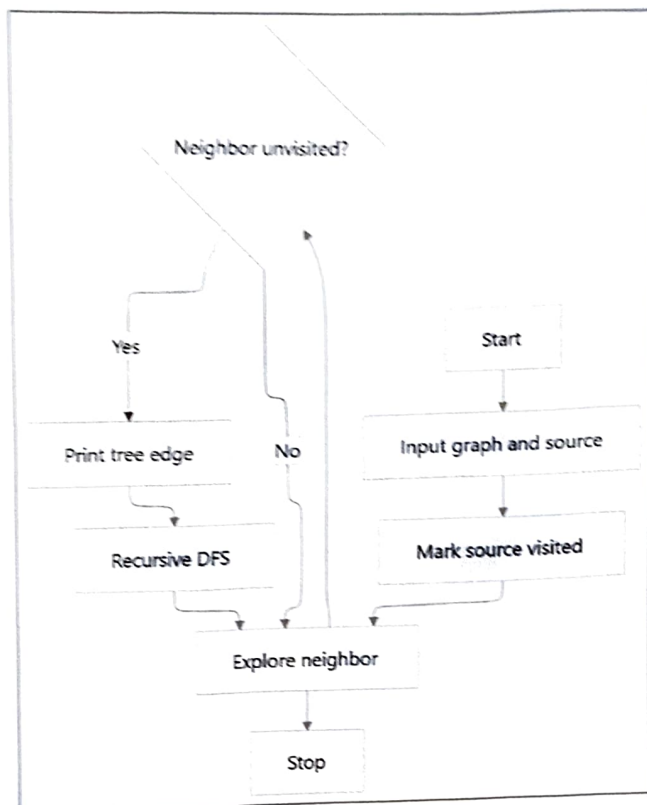
```
    return 0;
```

```
}
```

4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

Flowchart Diagram:



First Search (DFS):

Code:

```
#include <stdio.h>
```

```
int a[10][10], v[10], n;
```

```
void dfs (int x) {
```

```
    int i;
```

```
    v[x] = 1;
```

```
    for (i = 0; i < n; i++)
```

```
        if (a[x][i] && !v[i])
```

```
            printf ("%d\n", x, i);
```

```
            dfs (i);
```

```
}
```

```
}
```

```
int main () {
```

```
    int s, i, j;
```

```
    scanf ("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
```

```
            scanf ("%d", &a[i][j]);
```

```
    scanf ("%d", &s);
```

```
    dfs (s);
```

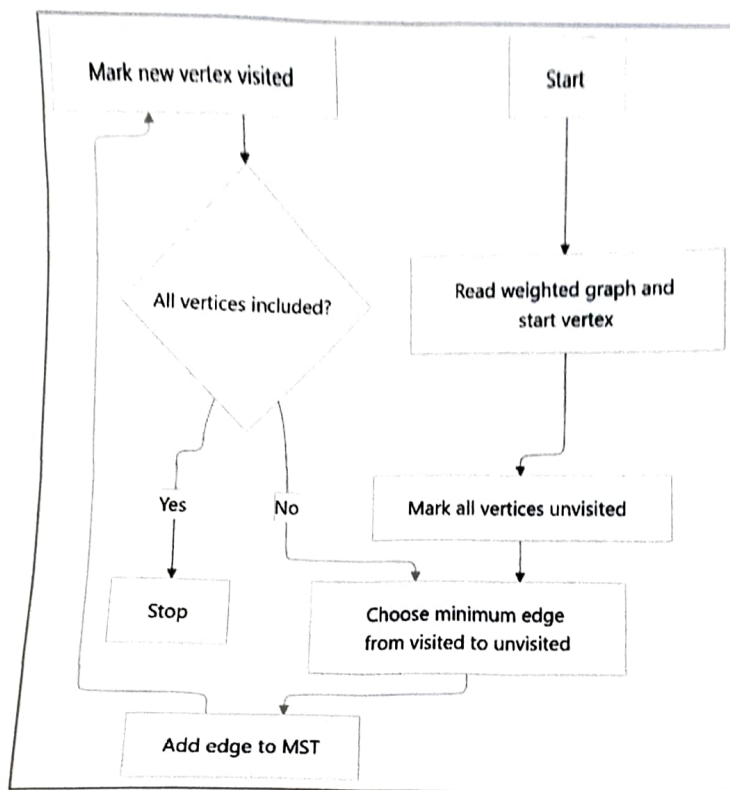
```
    return 0;
```

```
}
```

5. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

Flowchart Diagram:



Prim's Algorithm:

code:

```
#include <stdio.h>
```

```
#define INF 9999
```

```
int main ()
```

```
{
```

```
int n, a[10], w, v[10] = {0};
```

```
scanf ("%d", &n);
```

```
for (i=0; i<n; i++)
```

```
for (j=0; j<n; j++)
```

```
scanf ("%d", &a[i][j]);
```

```
scanf ("%d", &w);
```

```
v[0] = 1;
```

```
for (k=1; k<n; k++) {
```

```
min = INF
```

```
for (i=0; i<n; i++)
```

```
if (v[i])
```

```
for (j=0; j<n; j++)
```

```
if (v[j] && a[i][j] < min)
```

```
min = a[i][j] u=i, w=j;
```

```
printf ("%d - %d; %d\n", u, w, min);
```

```
v[w] = 1;
```

```
return 0;
```